Android Introduction

1

Android Devices

Android devices come in all shapes and sizes including, but not limited to …

- Smartphones
- Tablets
- E-reader devices
- Internet TVs
- Automobiles
- Smartwatches
- Smart Rings

2

Another List

1. **Watches**
2. **Smart glasses**
3. **Home Appliances**
4. **Cars**
5. **Homes**
6. **Cameras**
7. **Smart TVs**
8. **DECT phones**
9. **Game consoles**
10. **Mirrors ….**

Smart rings (and similar devices) generally do not run a distinct "smart ring" operating system; instead, they operate on a real-time operating system (RTOS) and communicate with companion apps on Android or iOS-based smartphones for managing data and features. Some specific models, like the Samsung Galaxy Ring, are restricted to Android devices, particularly Samsung phones

3

Vehicles use Android

- **Android Auto:** which mirrors your phone's Android interface onto the car's display
- **Android Automotive OS :** a full-fledged operating system built directly into the car's hardware that powers the infotainment system and allows for native app installation

4

Mobile Devices Makers

- https://www.gsmarena.com/makers.php3
- https://en.wikipedia.org/wiki/Android_Automotive

5

Vehicles Brands …

- **Abarth**
- **Acura**
- **Alfa Romeo**
- **Audi**
- …
- **Toyota**
- **Volkswagen**
- **Volvo**

https://www.android.com/auto/compatibility/#compatibility-vehicles

6

Project Ideas

- https://buildfire.com/best-app-ideas/
- https://nevonprojects.com/project-ideas/android-project-ideas/

7

Standard Features of Android

- Android is open source and freely available to manufacturers for customization, there are no fixed hardware or software configurations.
- **Storage**
- **Connectivity**
- **Media support**
- **Messaging**
- **Hardware support**
- **Multi-touch**
- **Tethering**

8

9

Version 16

- The latest released version of Android is Android 16, with a stable release date of June 10, 2025. Android 16 introduces features like enhanced support for hearing devices, and it is now rolling out to devices, with update schedules varying by manufacturer and device

10

Features

The base Android OS supports many features (Till 8.0), including

- **Storage (Database)**—SQLite, a lightweight relational database, for data storage.
- **Connectivity (CNC)**—GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth (includes A2DP and AVRCP), Wi-Fi, LTE, and WiMAX.
- **Media support** (Graphics) H.263, H.264 (in 3GP or MP4 container), MPEG-4 SP, AMR, AMR-WB (in 3GP container), AAC, HE-AAC (in MP4 or 3GP container), MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP.

11

Features

- **Messaging (CNC)** —Both SMS and MMS.
- **Hardware support (CA)**—Accelerometer sensor, camera, digital compass, proximity sensor, and GPS.
- **Multi-touch (HCI)**—Multi-touch screens.
- **Multi-tasking (OS)**—Multi-tasking applications.
- **Tethering (CNC/OS)**—Sharing of Internet connections as a wired/wireless hotspot.

12

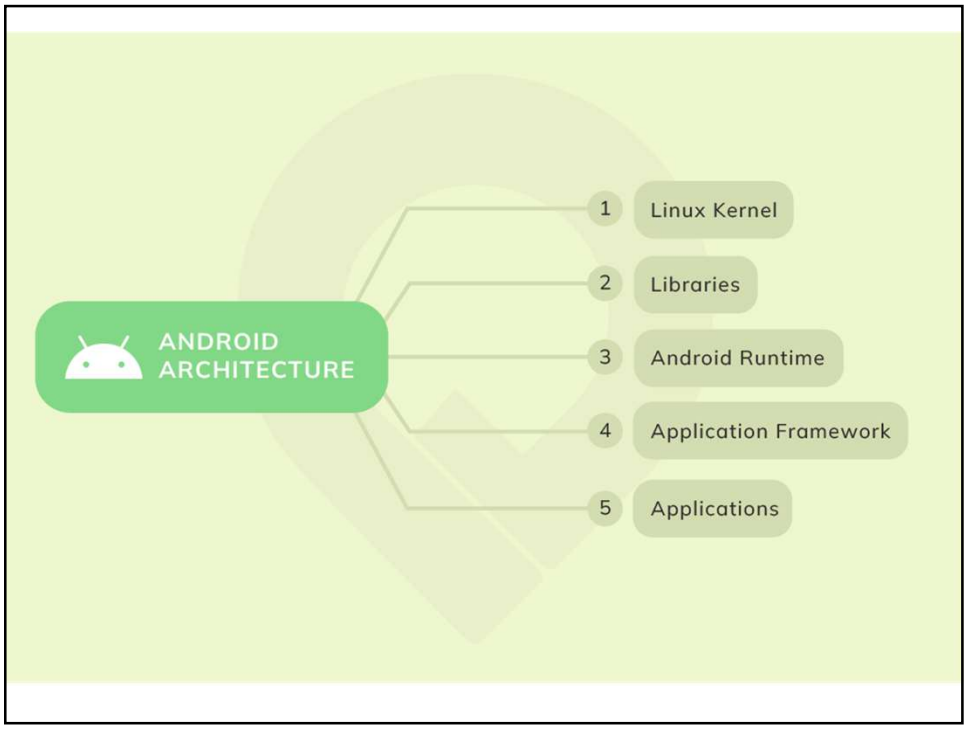
- https://www.android.com/new-features-on-android/

13

Window 1.1 (2023)

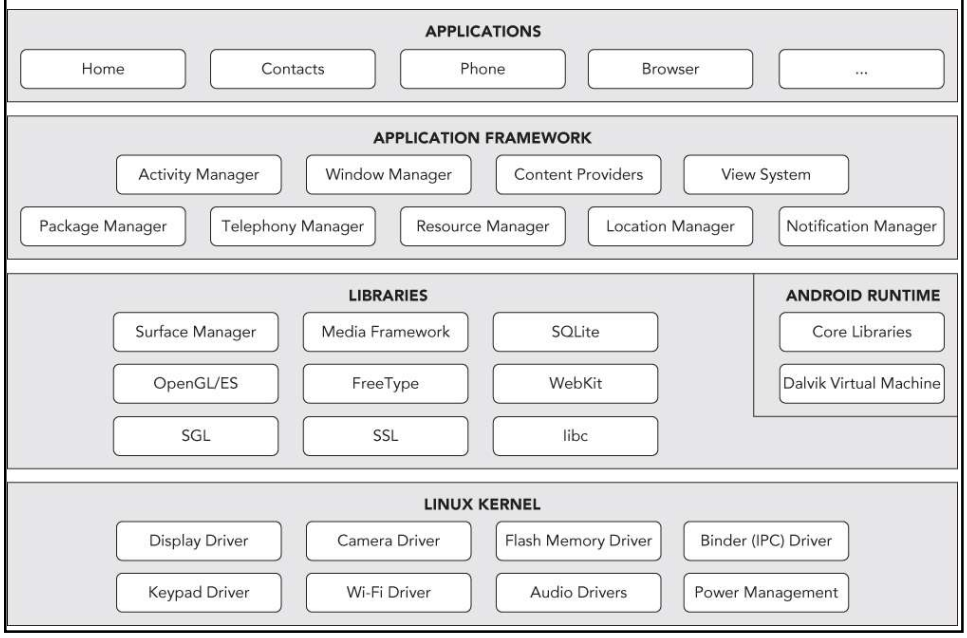
1.1 stabilizes activity embedding APIs, allowing apps like WhatsApp, eBay and Temu to ship large screen layouts. The API is enriched with features, as it enables developers to modify split screen behavior, check (and change) split state at runtime, implement horizontal splits, and start a modal in full window.

14



15

Architecture of Android - Old



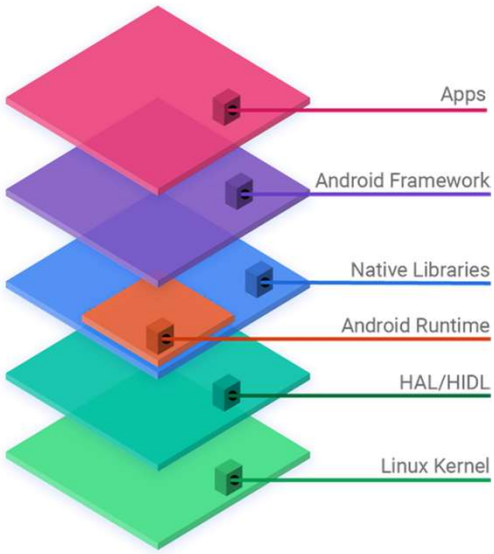
16

Storage

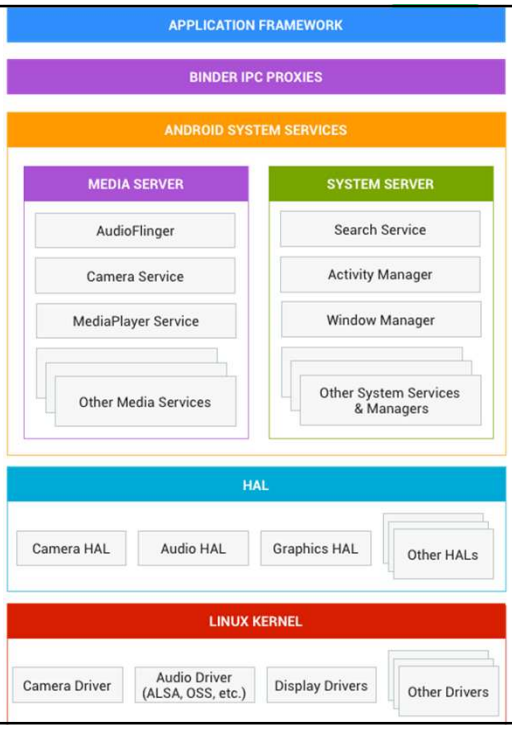
- Android uses a file system that's similar to disk-based file systems on other platforms. The system provides several options for you to save your app data:
- **App-specific storage:** Store files that are meant for your app's use only, either in dedicated directories within an internal storage volume or different dedicated directories within external storage. Use the directories within internal storage to save sensitive information that other apps shouldn't access.
- **Shared storage:** Store files that your app intends to share with other apps, including media, documents, and other files.
- **Preferences:** Store private, primitive data in key-value pairs.
- **Databases:** Store structured data in a private database using the Room persistence library. The Room persistence library provides an abstraction layer over SQLite to allow fluent database access while harnessing the full power of SQLite.

17

Android system architecture



18



19

Android System Architecture

- **Application framework**. The application framework is used most often by application developers. As a hardware developer, you should be aware of developer APIs as many map directly to the underlying HAL interfaces and can provide helpful information about implementing drivers.

20

- **Binder IPC**. The Binder Inter-Process Communication (IPC) mechanism allows the application framework to cross process boundaries and call into the Android system services code. This enables high level framework APIs to interact with Android system services. At the application framework level, this communication is hidden from the developer and things appear to "just work".

21

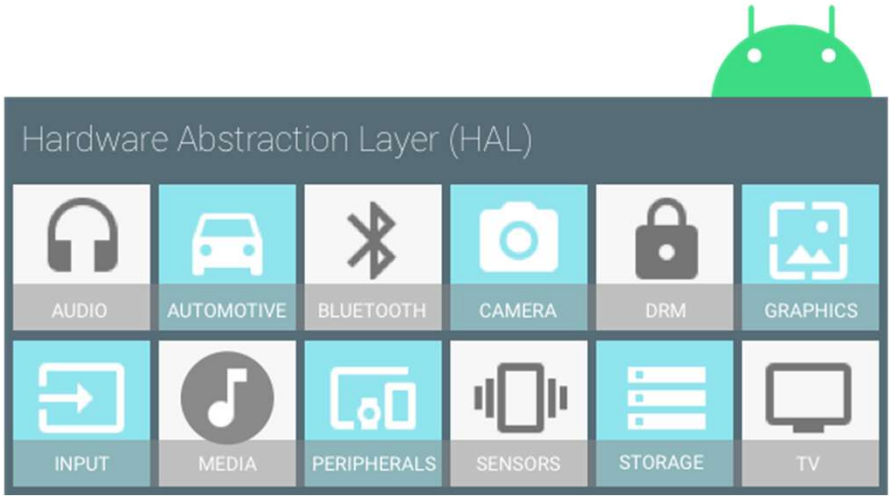
- **System services**. System services are modular, focused components such as Window Manager, Search Service, or Notification Manager. Functionality exposed by application framework APIs communicates with system services to access the underlying hardware. Android includes two groups of services: *system* (such as Window Manager and Notification Manager) and *media* (services involved in playing and recording media).

22

- **Hardware abstraction layer (HAL)**. A HAL defines a standard interface for hardware vendors to implement, which enables Android to be agnostic about lower-level driver implementations. Using a HAL allows you to implement functionality without affecting or modifying the higher level system. HAL implementations are packaged into modules and loaded by the Android system at the appropriate time.

23

Legacy HAL



24

Linux Kernel

- The foundation of the Android platform is the Linux kernel. For example, the Android Runtime (ART) relies on the Linux kernel for underlying functionalities such as threading and low-level memory management.
- Using a Linux kernel allows Android to take advantage of key security features and allows device manufacturers to develop hardware drivers for a well-known kernel.

25

- Developing your device drivers is similar to developing a typical Linux device driver.
- Android uses a version of the Linux kernel with a few special additions such as Low Memory Killer (a memory management system that is more aggressive in preserving memory), wake locks (a PowerManager system service), the Binder IPC driver, and other features important for a mobile embedded platform.
- Additions are primarily for system functionality and do not affect driver development.
- Any version of the kernel as long as it supports the required features (such as the binder driver).
- Recommended using the latest version of the Android kernel.

26

HAL interface definition language (HIDL)

- Android 8.0 re-architected the Android OS framework (in a project known as *Treble*) to make it easier, faster, and less costly for manufacturers to update devices to a new version of Android.
- In this new architecture, the HAL interface definition language (HIDL, pronounced "hide-l") specifies the interface between a HAL and its users, enabling the Android framework to be replaced without rebuilding the HALs.

27

HIDL

- HIDL separates the vendor implementation (device-specific, lower-level software written by silicon manufacturers) from the Android OS framework via a new vendor interface. Vendors or SOC makers build HALs once and place them in a /vendor partition on the device; the framework, in its own partition, can then be replaced with an over-the-air (OTA) update without recompiling the HALs.
- The difference between the legacy Android architecture and the current, HIDL-based architecture is in the use of the vendor interface

28

System Architecture resources

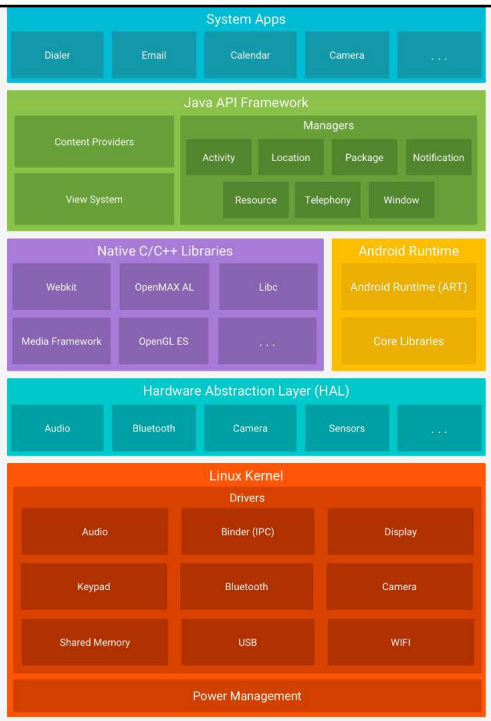
- For details on the Android architecture, see the following sections:
- HAL Types. Describes binderized, passthrough, Same-Process (SP), and legacy HALs.
- HIDL (General). Contains general information about the interface between a HAL and its users.
- HIDL (C++). Contains details for creating C++ implementations of HIDL interfaces.
- HIDL (Java). Contains details about the Java frontend for HIDL interfaces.
- ConfigStore HAL. Describes APIs for accessing read-only configuration items used to configure the Android framework.
- Device Tree Overlays. Provides details on using device tree overlays (DTOs) in Android.
- Vendor Native Development Kit (VNDK). Describes the set of vendor-exclusive libraries for implementing vendor HALs.
- Vendor Interface Object (VINTF). Describes the objects that aggregate relevant information about a device and make that information available through a queryable API.
- SELinux for Android 8.0. Details SELinux changes and customizations.

29

Platform Architecture

- Android is an open source, Linux-based software stack created for a wide array of devices and form factors. The following diagram shows the major components of the Android platform.

30



31

Hardware Abstraction Layer (HAL)

- The hardware abstraction layer (HAL) provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework. The HAL consists of multiple library modules, each of which implements an interface for a specific type of hardware component, such as the camera or bluetooth module. When a framework API makes a call to access device hardware, the Android system loads the library module for that hardware component.

32

Android Runtime

- For devices running Android version 5.0 (API level 21) or higher, each app runs in its own process and with its own instance of the Android Runtime (ART). ART is written to run multiple virtual machines on low-memory devices by executing DEX files, a bytecode format designed specially for Android that's optimized for minimal memory footprint. Build toolchains, such as Jack, compile Java sources into DEX bytecode, which can run on the Android platform.

33

- Some of the major features of ART include the following:
- Ahead-of-time (AOT) and just-in-time (JIT) compilation
- Optimized garbage collection (GC)
- On Android 9 (API level 28) and higher, conversion of an app package's Dalvik Executable format (DEX) files to more compact machine code.
- Better debugging support, including a dedicated sampling profiler, detailed diagnostic exceptions and crash reporting, and the ability to set watchpoints to monitor specific fields
- Prior to Android version 5.0 (API level 21), Dalvik was the Android runtime. If your app runs well on ART, then it should work on Dalvik as well, but the reverse may not be true.
- Android also includes a set of core runtime libraries that provide most of the functionality of the Java programming language, including some Java 8 language features, that the Java API framework uses.

34

Native C/C++ Libraries

- Many core Android system components and services, such as ART and HAL, are built from native code that require native libraries written in C and C++. The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to apps. For example, you can access OpenGL ES through the Android framework's Java OpenGL API to add support for drawing and manipulating 2D and 3D graphics in your app.
- If you are developing an app that requires C or C++ code, you can use the Android NDK to access some of these native platform libraries directly from your native code.

35

Java API Framework

- The entire feature-set of the Android OS is available to you through APIs written in the Java language. These APIs form the building blocks you need to create Android apps by simplifying the reuse of core, modular system components and services, which include the following:
- A rich and extensible View System you can use to build an app's UI, including lists, grids, text boxes, buttons, and even an embeddable web browser
- A Resource Manager, providing access to non-code resources such as localized strings, graphics, and layout files
- A Notification Manager that enables all apps to display custom alerts in the status bar
- An Activity Manager that manages the lifecycle of apps and provides a common navigation back stack
- Content Providers that enable apps to access data from other apps, such as the Contacts app, or to share their own data
- Developers have full access to the same framework APIs that Android system apps use.

36

System Apps

- Android comes with a set of core apps for email, SMS messaging, calendars, internet browsing, contacts, and more. Apps included with the platform have no special status among the apps the user chooses to install. So a third-party app can become the user's default web browser, SMS messenger, or even the default keyboard (some exceptions apply, such as the system's Settings app).
- The system apps function both as apps for users and to provide key capabilities that developers can access from their own app. For example, if your app would like to deliver an SMS message, you don't need to build that functionality yourself—you can instead invoke whichever SMS app is already installed to deliver a message to the recipient you specify.

37

Linux Security

- At the operating system level, the Android platform provides the security of the Linux kernel, as well as a secure inter-process communication (IPC) facility to enable secure communication between applications running in different processes. These security features at the OS level ensure that even native code is constrained by the Application Sandbox. Whether that code is the result of included application behavior or an exploitation of an application vulnerability, the system is designed to prevent the rogue application from harming other applications, the Android system, or the device itself.

- Kernel Configuration for measures you can take to strengthen the kernel on your devices.
- Android Compatibility Definition Document (CDD) for required settings.
- https://source.android.com/security/overview/kernel-security.html

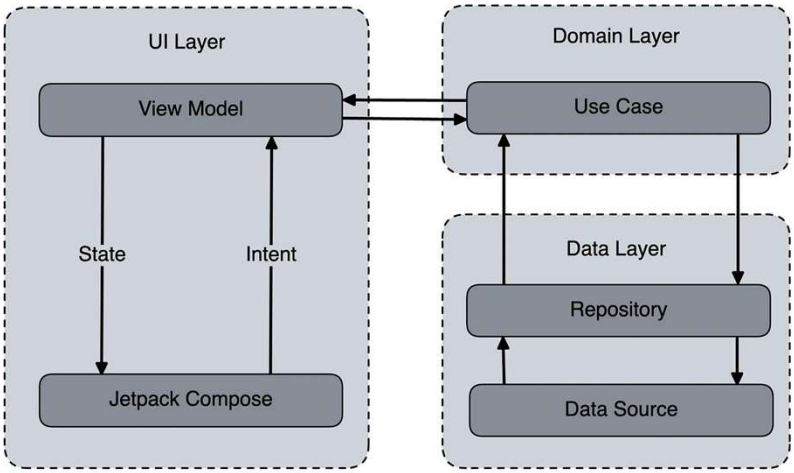
38

Security

- The foundation of the Android platform is the Linux kernel. The Linux kernel has been in widespread use for years, and is used in millions of security-sensitive environments. Through its history of constantly being researched, attacked, and fixed by thousands of developers, Linux has become a stable and secure kernel trusted by many corporations and security professionals.
- As the base for a mobile computing environment, the Linux kernel provides Android with several key security features, including:
- A user-based permissions model
- Process isolation
- Extensible mechanism for secure IPC
- The ability to remove unnecessary and potentially insecure parts of the kernel
- As a multiuser operating system, a fundamental security objective of the Linux kernel is to isolate user resources from one another. The Linux security philosophy is to protect user resources from one another. Thus, Linux:
- Prevents user A from reading user B's files
- Ensures that user A does not exhaust user B's memory
- Ensures that user A does not exhaust user B's CPU resources
- Ensures that user A does not exhaust user B's devices (for example, telephony, GPS, and Bluetooth)

39

App Architecture… uses **MVI (Model View Intent)** architecture pattern



40

The activity lifecycle

- As a user navigates through, out of, and back to your app, the Activity instances in your app transition through different states in their lifecycle.
- The Activity class provides a number of callbacks that let the activity know when a state changes or that the system is creating, stopping, or resuming an activity or destroying the process the activity resides in.
- Within the lifecycle callback methods, you can declare how your activity behaves when the user leaves and re-enters the activity.

For example, if you're building a streaming video player, you might pause the video and terminate the network connection when the user switches to another app. When the user returns, you can reconnect to the network and let the user resume the video from the same spot.

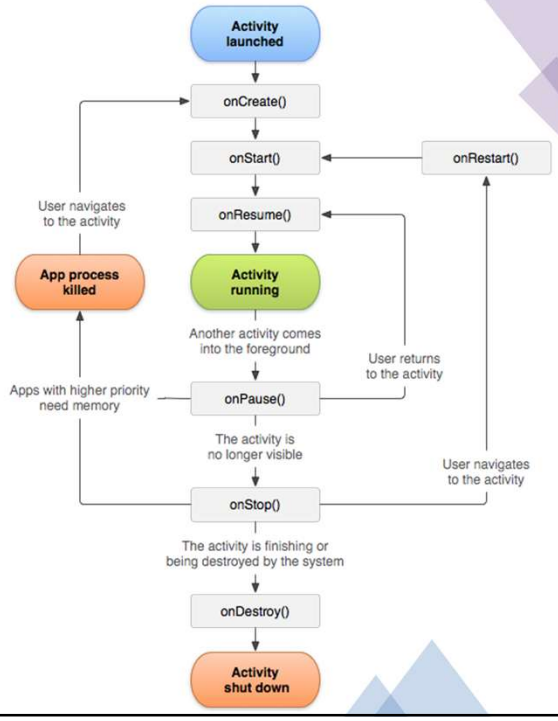
41

Callbacks

For example, good implementation of the lifecycle callbacks can help your app avoid the following:

- Crashing if the user receives a phone call or switches to another app while using your app.
- Consuming valuable system resources when the user is not actively using it.
- Losing the user's progress if they leave your app and return to it at a later time.
- Crashing or losing the user's progress when the screen rotates between landscape and portrait orientation.

42



43

Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.
onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.

44

Activity-lifecycle concepts

To navigate transitions between stages of the activity lifecycle,
the Activity class provides a core set of six callbacks:
onCreate(), onStart(), onResume(),
onPause(), onStop(), and onDestroy().
The system invokes each of these callbacks as the activity enters a new state.

In Android development, a callback function, often simply called a "callback," is a method that is passed as an argument to another method and is executed at a later time, usually when a specific event occurs or a task completes. This mechanism allows for asynchronous communication and event handling within an application

45

Leaving activity

- As the user begins to leave the activity, the system calls methods to dismantle the activity. In some cases, the activity is only partially dismantled and still resides in memory, such as when the user switches to another app. In these cases, the activity can still come back to the foreground.
- If the user returns to the activity, it resumes from where the user left off. With a few exceptions, apps are restricted from starting activities when running in the background.

46

Killing Process

- The system's likelihood of killing a given process, along with the activities in it, depends on the state of the activity at the time.
- Depending on the complexity of your activity, you probably don't need to implement all the lifecycle methods. However, it's important that you understand each one and implement those that make your app behave the way users expect.

47

Callback Methods

- Some actions belong in the activity lifecycle methods. However, place code that implements the actions of a dependent component in the component, rather than the activity lifecycle method. To achieve this, you need to make the dependent component lifecycle-aware. To learn how to make your dependent components lifecycle-aware,
- Handling Lifecycles with Lifecycle-Aware Components.

48

onCreate()

You must implement this callback, which fires when the system first creates the activity. On activity creation, the activity enters the *Created* state. In the onCreate() method, perform basic application startup logic that happens only once for the entire life of the activity.

49

onStart()

When the activity enters the Started state, the system invokes onStart(). This call makes the activity visible to the user as the app prepares for the activity to enter the foreground and become interactive. For example, this method is where the code that maintains the UI is initialized.

When the activity moves to the Started state, any lifecycle-aware component tied to the activity's lifecycle receives the ON_START event.

The onStart() method completes quickly and, as with the Created state, the activity does not remain in the Started state. Once this callback finishes, the activity enters the *Resumed* state and the system invokes the onResume() method

50

onResume()

When the activity enters the Resumed state, it comes to the foreground, and the system invokes the onResume() callback.
This is the state in which the app interacts with the user.
The app stays in this state until something happens to take focus away from the app, such as the device receiving a phone call, the user navigating to another activity, or the device screen turning off.
When the activity moves to the Resumed state, any lifecycle-aware component tied to the activity's lifecycle receives the ON_RESUME event.
This is where the lifecycle components can enable any functionality that needs to run while the component is visible and in the foreground, such as starting a camera preview.
When an interruptive event occurs, the activity enters the *Paused* state and the system invokes the onPause() callback.

51

onPause()

- The system calls this method as the first indication that the user is leaving your activity, though it does not always mean the activity is being destroyed. It indicates that the activity is no longer in the foreground, but it is still visible if the user is in multi-window mode. There are several reasons why an activity might enter this state:
- An event that interrupts app execution, as described in the section about the onResume() callback, pauses the current activity. This is the most common case.
- In multi-window mode, only one app has focus at any time, and the system pauses all the other apps.
- The opening of a new, semi-transparent activity, such as a dialog, pauses the activity it covers. As long as the activity is partially visible but not in focus, it remains paused.

52

onPause()

When an activity moves to the Paused state, any lifecycle-aware component tied to the activity's lifecycle receives the ON_PAUSE event. This is where the lifecycle components can stop any functionality that does not need to run while the component is not in the foreground, such as stopping a camera preview.

Use the onPause() method to pause or adjust operations that can't continue, or might continue in moderation, while the Activity is in the Paused state, and that you expect to resume shortly.

You can also use the onPause() method to release system resources, handles to sensors (like GPS), or any resources that affect battery life while your activity is Paused and the user does not need them.

53

onPause()

onPause() execution is very brief and does not necessarily offer enough time to perform save operations. For this reason, **don't** use onPause() to save application or user data, make network calls, or execute database transactions. Such work might not complete before the method completes.

Instead, perform heavy-load shutdown operations during onStop().

54

onStop()

When your activity is no longer visible to the user, it enters the *Stopped* state, and the system invokes the onStop() callback. This can occur when a newly launched activity covers the entire screen.
The system also calls onStop() when the activity finishes running and is about to be terminated.
When the activity moves to the Stopped state, any lifecycle-aware component tied to the activity's lifecycle receives the ON_STOP event. This is where the lifecycle components can stop any functionality that does not need to run while the component is not visible on the screen.

55

onStop()

In the onStop() method, release or adjust resources that are not needed while the app is not visible to the user. For example, your app might pause animations or switch from fine-grained to coarse-grained location updates. Using onStop() instead of onPause() means that UI-related work continues, even when the user is viewing your activity in multi-window mode.
Also, use onStop() to perform relatively CPU-intensive shutdown operations. For example, if you can't find a better time to save information to a database, you might do so during onStop().

56

onDestroy()

onDestroy() is called before the activity is destroyed. The system invokes this callback for one of two reasons:

1.The activity is finishing, due to the user completely dismissing the activity or due to finish() being called on the activity.

2.The system is temporarily destroying the activity due to a configuration change, such as device rotation or entering multi-window mode.

When the activity moves to the destroyed state, any lifecycle-aware component tied to the activity's lifecycle receives the ON_DESTROY event. This is where the lifecycle components can clean up anything they need to before the Activity is destroyed.

57

Activity state and ejection from memory

- The system kills processes when it needs to free up RAM. The likelihood of the system killing a given process depends on the state of the process at the time. Process state, in turn, depends on the state of the activity running in the process. Table shows the correlations among process state, activity state, and the likelihood of the system killing the process. This table only applies if a process is not running other types of application components.

58

Likelihood of being killed	Process state	Final activity state
Lowest	Foreground (having or about to get focus)	Resumed
Low	Visible (no focus)	Started/Paused
Higher	Background (invisible)	Stopped
Highest	Empty	Destroyed

59